



UNIVERSIDAD DE CÓRDOBA

GRADO EN INGENIERÍA INFORMÁTICA
SISTEMAS EN TIEMPO REAL



UNIVERSIDAD DE CÓRDOBA

Autores:

Álvaro Palacios Criado p02pacra@uco.es

Profesor

José Manuel Palomares Muñoz

Córdoba, 13 de marzo de 2023

Índice

1. Estructura de los Sistemas de Control	2
1.1. Sistemas Centralizados	2
1.2. Sistemas Jerárquicos	2
1.3. Sistemas Distribuidos	3
2. Computadores, Microcontroladores y Procesadores Especializados	4
2.1. Microprocesadores	4
2.2. Microcontroladores	4
2.3. Procesadores Especializados	5
3. Temporizador	5
3.1. Reloj de tiempo real	5
3.2. Watchdog timer	5
4. Técnicas de Transferencia de Datos	6
4.1. Muestreo (Polling)	6
4.2. Interrupciones	7
4.2.1. "Daisy Chain"	9
4.2.2. Codificador de prioridad	9
4.3. Acceso directo a Memoria (DMA)	11
4.4. Comparación entre las distintas técnicas	11

Índice de figuras

1. Esquema de un Sistema Centralizado	2
2. Esquema de un Sistema Jerárquico	3
3. Esquema de Sistema Distribuido	3
4. Estructura de watchdog timer	5
5. Esquema de un control directo	6
6. Mecanismos Polling	6
7. Esquema de flujo de Interrupciones	7
8. Señales de CPU para las interrupciones	8
9. Estructura de pines de un dispositivo	9
10. Estructura de codificador de prioridad	10
11. Codificador con prioridad con enmascaramiento	10

1. Estructura de los Sistemas de Control

1.1. Sistemas Centralizados

Idea original → 1 único computador controla todo el subsistema de una planta.

El tiempo medio entre fallos del hardware estaba en el orden de muy pocas horas. Un fallo provocaba una parada de la planta: software muy restrictivo para evitar fallos debido el software, ya que el hardware falla mucho. Los computadores tenían válvulas de vacío que emitían mucho calor, con lo que se utilizaba un edificio anexo para disipar el calor. (Consumo del orden de KW/h)

A finales de los 60, se reduce el tamaño de las válvulas de vacío y dividir a la mitad el consumo. Aparecen más fabricantes decae el precio → se formaron los subsistemas duales (dos ordenadores), esquema dúplex en reserva. Aumenta muchísimo la eficiencia.

En los años 70 se comienzan los transistores y a construir los microprocesadores. Los sistemas centralizados en una manera mas minoritario a nivel industrial y en plantas de muy pocos subsistemas.

Esquema sistema centralizado (figura 1) → este esquema necesita poca distancia entre los sensores-entrono y sensores-CPU, requiere suficiente potencia.

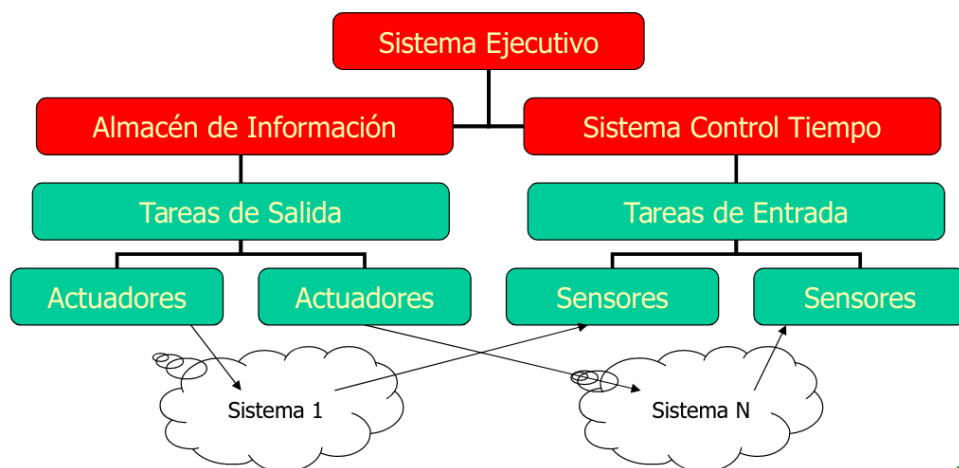


Figura 1: Esquema de un Sistema Centralizado

1.2. Sistemas Jerárquicos

Comienzan los sistemas distribuidos, lo que permite poner cada sensor con un hardware acorde con su consumo, por ejemplo un sensor de temperatura bastaría con un Arduino.

Los sistemas jerárquicos siguen la estructura de toma de decisiones de una gran empresa.

Estructura piramidal:

- Elemento toma las decisiones de alto nivel para problemas muy complejos que involucren muchos subsistemas.
- Elemento toma las decisiones de bajo nivel para problemas muy simples.

Esto permite una balanceo de carga. Puede haber problemas por bloqueo de los elementos que toman las decisiones de alto nivel.

Para comunicación un nodo de mismo nivel con otro de mismo nivel tiene que comunicarlo por los nodos superiores. Si falla un nodo superior que se involucre en la comunicación, se produce un fallo en la misma. Pero esto no bloquea al sistema. (figura 2)

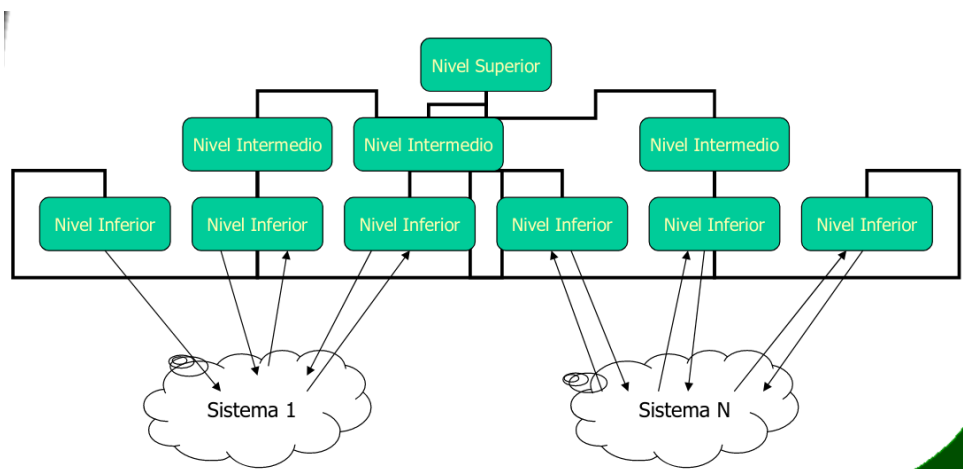


Figura 2: Esquema de un Sistema Jerárquico

1.3. Sistemas Distribuidos

Estos sistemas intentan solucionar el bloqueo que se producían en los sistemas jerárquicos. Se basan en muchos pequeños elementos de cómputo interconectados, con capacidades similares y responsabilidades de toma de decisión similares. De esta manera ante una caída no hay bloqueo. Estos elementos deben ser reconfigurables, lo que permiten adaptarse a situaciones muy cambiantes, aumentar o disminuir unidades, reasignar y dividir tareas, etc.

Estos nodos deben estar conectados, el trabajo se divide entre los diferentes elementos, tienen alta redundancia ante fallos, elevada dificultad, mayor complejidad de hardware y de software.

Hay un problema, como cada nodo puede ejecutar la tarea o el sensor que consume mayor recurso (Líder), todos tienen que tener las mismas prestaciones. Hay que tener en cuenta que si hay que medir algo en lo que el sensor tiene que estar cerca tenemos un problema, por eso se mezcla el jerárquico y el distribuido. La parte que tiene que medir algo físico sigue un modelo jerárquico (cercanía del sensor), y la toma de decisión debida a esa medida se hace con un sistema distribuido (nube). (figura 3). De esto tenemos un ejemplo claro las redes P2P.

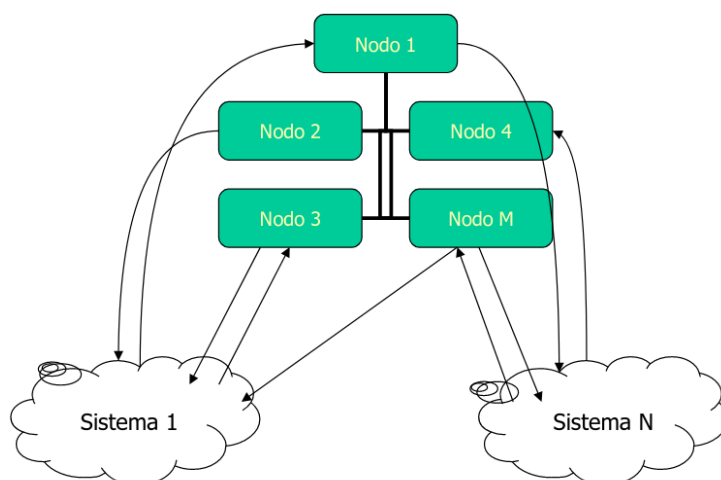


Figura 3: Esquema de Sistema Distribuido

2. Computadores, Microcontroladores y Procesadores Especializados

2.1. Microprocesadores

Se utilizan en sistemas de propósito general → para cualquier tarea, como en los Sistemas Distribuidos.

El rendimiento del procesador viene dado por (Siempre para cumplir con el Deadline):

- Potencia de procesado.
- Ancho de banda E/S (Cuello de botella).
- Estructura de interrupciones (Si hay muchos fallos perderemos el Deadline).
- Capacidad de Almacenamiento Masivo.

Ventajas:

- Alta potencia de cálculo.
- Versatilidad.

Inconvenientes:

- Alto coste.
- Necesita más hardware para funcionar.
- Altas necesidades de consumo energético.

2.2. Microcontroladores

Procesadores que tienen integrados dentro del mismo chip todos los elementos que componen un computador: CPU, memoria e interfaces E/S. Incluyen elementos específicos para el control de sistemas. Conversores A/D, D/A, temporizadores, puertos serie/paralelo.

Principales diferencias arquitecturales:

- Operaciones de manipulación de bit.
- Rápido reconocimiento y procesamiento de interrupciones.
- Operaciones de memoria simples rápidas → RAM interna y algunos registros mapeados en memoria.
- Alta velocidad de cambio de contexto. Ej: banco de registros.

Ventajas:

- Integración completa de un sistema: poco hardware adicional.
- Coste reducido.
- Bajo consumo energético.

Inconvenientes:

- Capacidad computacional reducida.
- Menor número de herramientas software gratuitas disponibles.
- Necesidad de un sistema "anfitrión" para desarrollar el sistema externamente.

2.3. Procesadores Especializados

Son procesadores que realizan algunas operaciones o tareas específicas de alta complejidad o para ejecutar algunas tareas críticas. (Arquitecturas RISC → mayor facilidad).

Para aumentar el rendimiento hay que abandonar la filosofía de Von Neumann: DSP, procesadores específicos: codiseño HW/SW, SoC, etc.

Ventajas: Permite ajustarse a requisitos temporales muy exigentes.

Inconvenientes:

- Muy caros.
- Pocas herramientas software de soporte.
- Hay que construir todo el software de sistema: drivers, etc.

3. Temporizador

3.1. Reloj de tiempo real

Puede ser muy complejo o tan simple y mínimo como un generador de pulsos. Suelen tener una alta precisión.

Sirven para medir el tiempo transcurrido (en ticks de reloj) con muy bajo error de estimación. Suelen tener asociada una señal de interrupción para que el sistema ejecutivo puede tener un control exacto del tiempo transcurrido (segundos).

No tiene que coincidir el tiempo del ordenador y el tiempo Humano.

3.2. Watchdog timer

Consiste en un contador que se incrementa con cierta frecuencia, pudiendo reiniciarlo y ponerlo a 0 (figura 4).

El desbordamiento se produce en aquellos casos en los que se sobrepase un cierto límite temporal antes de que se haya reiniciado el contador.

Con esto evitamos los bucles infinitos. La señal de desbordamiento suele estar enlazada con la señal de RESET del procesador o simplemente conectada a una interrupción que se procesa.

Mecanismo simple para detectar lazos muertos, bucles infinitos y restaurar el sistema a un estado válido, permitiendo a la CPU continuar su procesamiento. En un bucle infinito poner doble condición, una siempre con el tiempo en comparación con el Deadline, de esta manera los evitamos.

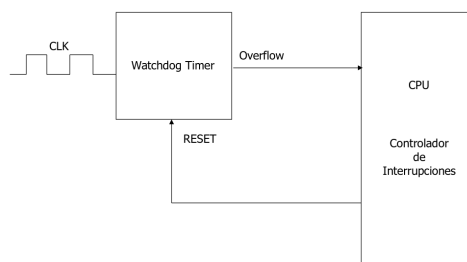


Figura 4: Estructura de watchdog timer

4. Técnicas de Transferencia de Datos

Todas las tareas de STR involucran transferencias de datos entre la CPU y dispositivos externos (sensores o actuadores). Deben ser síncronas con la CPU. Pero tenemos diferentes velocidades CPU $\longleftrightarrow$ dispositivo externo.

Hay poca eficiencia en casos de control de estos dispositivos directamente con la CPU $\rightarrow$ muchos ciclos de espera.

4.1. Muestreo (Polling)

Es una técnica muy rápida a velocidad de CPU. La CPU se encarga de comprobar un valor mediante un bucle que es modificado externamente (Control directo, figura 5). Hay dos mecanismos principalmente:

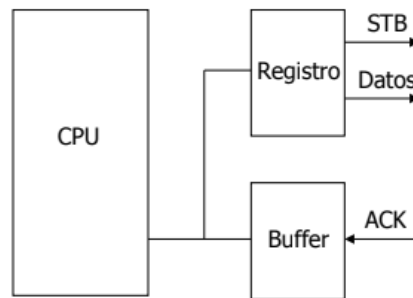


Figura 5: Esquema de un control directo

- Espera ocupada (busy wait) (Pizza-Mundial-Horno). (figura6a)
- Transferencia condicional (no bloqueante) (figura 6)



(a) Esquema de Espera Ocupada



(b) Esquema de Espera Condicional

Figura 6: Mecanismos Polling

4.2. Interrupciones

Es un mecanismo que permite interrumpir temporalmente el flujo de un programa para ejecutar un trozo de otro programa y continuar la ejecución de un programa original de manera transparente a éste. Escucha señales eléctricas y en un momento de ejecución asíncrona, salta y se va a una SRI asociada → ejecuta y vuelve.

Este mecanismo permite sincronizar dispositivos lentos mediante un control delegado en un procesador específico la gestión mientras que la CPU continua la ejecución de otras tareas. Controlan otras señales hardware:

- Reloj de tiempo real.
- Alarmas de sensores.
- Mantenimiento del sistema.
- Fallo en el hardware.
- Fallo en la alimentación.
- Soporte de depuración.
- Entrada al SO.

Tenemos un cambio de contexto transparente, para ello se almacenan los registros de la CPU y al terminar de ejecutarse la RTI se restauran los registros de la CPU con el valor almacenado (salvaguardado), tenemos que responder:

- **¿Qué se almacena? y ¿Cómo se almacena?:** todos los registros es muy ineficiente, con lo que se guarda el PC y flags (el resto de registros es trabajo del programador). No almacenar ningún registro si no se va a volver (Ej: fallo de alimentación, proceso destructivo)
- **¿Cómo regresar?:** el programador es el encargado de restaurar los registros almacenados antes de retornar de la RTI. (Restauración en orden inverso al almacenamiento → Estructura Pila(FIFO))
- **¿Dónde se almacena?:**
 - Grabar contenido en zona especial de memoria (se puede proteger).
 - Grabar contenido en la pila (cada proceso tiene su pila).
 - Tener varios bancos de registros auxiliares (si tienes 3 bancos solo puede hacer 2 llamadas ya que 1 esta en el main).

Aspectos especiales: si se desea imposibilitar otras interrupciones simultáneas, desactivar las interrupciones (CLI → pone a 0 el flag de interrupción). Antes de salir volver a habilitarlas (STI). Retornar mediante el retorno especial de interrupciones (IRET) (figura 7)

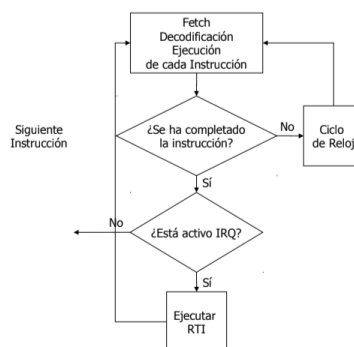


Figura 7: Esquema de flujo de Interrupciones

Proceso para inhabilitar las interrupciones:

- CPU con 2 señales de interrupción (figura 8):
 - IRQ: habilitar/deshabilitar (por software)
 - MMI: no se puede deshabilitar

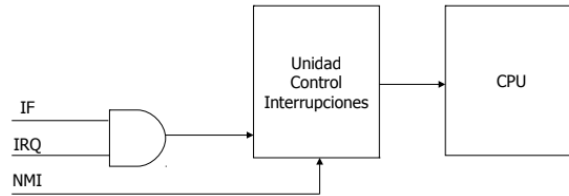


Figura 8: Señales de CPU para las interrupciones

Mecanismos de respuesta a una interrupción:

- ¿Cómo determinar que RTI se ejecuta?:
 1. Transferir el control a una dirección fija.
 2. Cargar el PC con una dirección contenida en un registro o palabra de memoria.
 3. General señal de salida para capturar parte de la dir de memoria donde esta la dirección de RTI.
 4. General señal de salida para capturar una instrucción dada por un dispositivo externo
- En las 2 primeras, el hardware solo informa que se ha producido una interrupción, debe determinarse cual vía software.
- En las 2 últimas, el hardware externo identificara la interrupción y saltará directamente a la RTI asociada

Interrupciones no determinadas por el hardware:

- Software determina cual ha sido la causa de la interrupción.
- Se chequea mediante Polling, ya que al saber que hay al menos una señal activa es lo más eficiente.
- Si hay más de una ya no es tan eficiente. Otros mecanismos:
 - Chequeo de las interrupciones más habituales primero.
 - Chequear interrupciones que requieran una respuesta rápida.
 - Si todas tiene el mismo tiempo, se rota el orden de chequeo para garantizar que se atienden todas uniformemente.
- Usando polling el método de prioridad puede ser tan flexible como se desee (software)

Interrupciones vectorizadas por hardware:

- El hardware es el encargado de informar qué causa produjo la interrupción.
- Las interrupciones están asociadas a un dispositivo externo y tienen asignados mecanismos de acceso a las RTI asociadas.
- Necesidad de un mecanismo de arbitrio y asignación de prioridades entre distintas fuentes de interrupción: “Daisy Chain”. y Codificador de prioridad.

4.2.1. "Daisy Chain"

Estructura es:

- Existen varias modalidades de 2,3 y 4 pines. Cada dispositivo tiene asociado 4 pines (figura 9):
 - 1 pin de solicitud de IRQ [Entrada externa]
 - 1 pin de solicitud de petición de ejecución RTI
 - 2 pines para la estructura Daisy Chain: IEI (Interrupt Enable In) y IEO (Interrupt Enable Out).
- IEO estará activo si: IEI lo esta, y dicha unidad no tiene IRQ activa.
- Si un dispositivo está solicitado interrupción:
 - Si su IEI esta activo → IEO pasa a inactivo hasta que finalice la ejecución de la RTI para impedir la ejecución de Interrupciones de menor prioridad.
 - Si su IEI esta inactivo → Esperar hasta que se active.

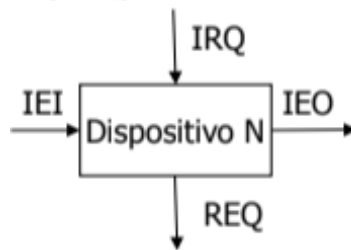


Figura 9: Estructura de pines de un dispositivo

Funcionamiento:

1. Cuando un dispositivo recibe una solicitud de interrupción, IRQ se activa.
2. El dispositivo solicita el control activando el bus común REQ.
3. El árbitro recibe la solicitud y activa la salida GRANT (IEI del primer dispositivo):
 - Si GRANT ya estaba activa, la desactiva una iteración y la activa después.
 - Si GRANT no está activa, la activa
4. Si el primer dispositivo no está solicitando la interrupción, activará IEO hasta que el dispositivo que lo solicite la reciba en cadena.
5. Cuando se le concede al dispositivo solicitante, desactiva el REQ. El árbitro no desactiva la salida GRANT de activación.
6. Cuando el dispositivo finaliza su rutina, desactiva IRQ y el árbitro desactiva GRANT.

Para las prioridades viene determinadas por la posición en la lista. La prioridad está fijada previamente y no puede ser modificada vía software. Hay que tener mucho cuidado con los tiempo de propagación y con la inhibición de interrupciones para poder garantizar que se garantizan las prioridades.

4.2.2. Codificador de prioridad

Estructura es (figura 10):

- Se solicita interrupción al sistema ante cualquiera de las N líneas que se activen y se genera un código de M bits especificando el número de la línea de interrupción activa.
- En el caso de que haya más de una línea activa el codificador con prioridad generará el código de una de las líneas de acuerdo con la prioridad preestablecida.
- Cuando se produce el reconocimiento de la interrupción se leerá por el bus de datos un valor que será distinto dependiendo de la interrupción más prioritaria que esté pedida.

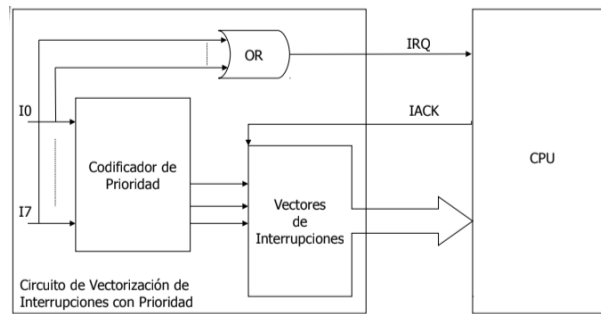


Figura 10: Estructura de codificador de prioridad

Vectores de respuesta a las interrupciones, el vector puede ser:

- Una instrucción (habitualmente JMP/CALL o IRET)
- La dirección de la RTI
- La dirección donde está almacenado el puntero a la RTI
- Parte de la dirección de la RTI
- Parte de la dirección donde está almacenado el puntero a la RTI

Interrupciones multinivel:

- En los STR no es admisible las interrupciones en un único nivel de prioridad.
- Se debe poder interrumpir el procesamiento de una interrupción por otra interrupción de más alta prioridad.
- Sin embargo, se debe impedir que una interrupción de menor nivel pueda interrumpir a otra con mayor prioridad: Enmascaramiento.

Enmascaramiento de Interrupciones (figura 11):

- Utilizando la estructura “Daisy Chain” el enmascaramiento de las interrupciones es automático y se produce al bloquear un dispositivo de alta prioridad todos los de baja prioridad (desactivando su IEO).
- Utilizando la estructura codificación de prioridades es mucho más versátil, ya que utilizando un registro de máscara podemos inhibir cualquier interrupción (no solo las prioritarias, sino cualquiera).
- Permite desactivar la interrupción asociada a una alarma: una vez se reconozca, ya no es necesario volver a tratarla y así no se consume tiempo de procesamiento.

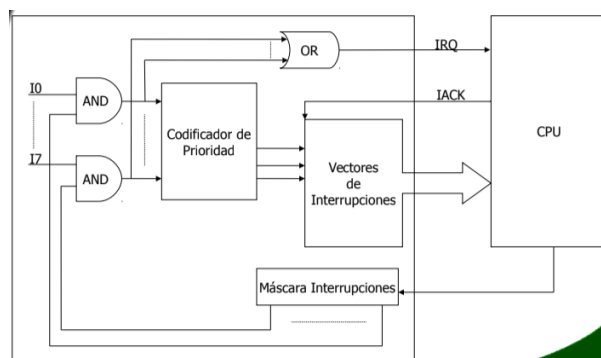


Figura 11: Codificador con prioridad con enmascaramiento

Rutina de Tratamiento de la Interrupción (RTI)

- Deshabilitar interrupciones
- Guardar el entorno de trabajo y registro de máscaras
- Colocar el nuevo registro de enmascaramiento
- Habilitar interrupciones
- Ejecutar la RTI en sí
- Deshabilitar las interrupciones
- Colocar el registro de enmascaramiento antiguo
- Colocar los registros almacenados previamente
- Habilitar de nuevo las interrupciones
- Retornar de la RTI

4.3. Acceso directo a Memoria (DMA)

Consiste en transferir datos desde/a el dispositivo E/S directamente a la memoria principal sin pasar por CPU. También permite mover/copiar bloques de memoria de una zona a otra de la memoria $\rightarrow$. El controlador DMA es un procesador dedicado y eficiente, ya que evita pasar por la CPU y permite que esta se dedique a otras tareas.

Para que el DMA pueda iniciar las transferencias, la CPU debe ceder el "*mastering*" de los buses:

- CPU cede el control de los buses hasta que el DMA haya finalizado todas las transferencias $\rightarrow$ Puede provocar problemas de Tpo Real.
- CPU cede el control de los buses y puede recuperarlos solicitándolos al DMA antes de que éste finalice completamente.

Hay 3 técnicas:

- Modo ráfaga ("burst mode") El controlador DMA solicita los buses a la CPU y realiza la transferencia completa. Puede afectar al tiempo de respuesta del sistema en el caso que se produzca un evento que deba de ser atendida inmediatamente.
- Modo distribuido: El controlador DMA solicita los buses a la CPU para hacer una única transferencia (de uno o varios bytes) y cede el control rápidamente a la CPU. Solo afectaría a STR que utilice "busy waiting" para el cálculo de tiempo.
- Modo oculto ("cycle-stealing"): El controlador DMA toma el control de los buses cuando la CPU no está siendo utilizada. Es la que menos afecta al tiempo del sistema, pero es la más lenta y delicada. Necesita Hardware especial que detecte esos ciclos "ociosos".

4.4. Comparación entre las distintas técnicas

- El "polling" es el método más sencillo para transferir datos en términos de facilidad de programación y chequeo del programa.
- El mecanismo de interrupciones es más eficiente que el "polling". Pero los programas son menos estructurados. Son difíciles de chequear (los errores dependen del tiempo). Las interrupciones son el método más eficaz para dispositivos que no transfieren grandes bloques de datos de una vez.
- Si se necesitan altas velocidades de transferencia, las interrupciones no son eficientes: se producen muchos tiempos muertos entre que se produce la interrupción hasta que se ejecuta la rutina de interrupción. En